



第六章 分治法（二）

朱容波

Email: rongbozhu@126.com

Office: 一综B323

华中农业大学信息学院



本章主要内容

- 分治基本概念
 - 引言
 - 两个简单例子
 - 二分搜索
 - 合并排序
 - 分治范式
- 分治策略解决的一些问题
 - 选择算法
 - 快速排序
 - 大整数乘法
 - 矩阵乘法
 - 最近点问题

华中农业大学信息学院



6.1 引言

? 什么是分治算法形式?

- 一个分治算法把问题实例划分成若干子实例(多数情况是分成两个);
- 并分别递归地解决每个子实例;
- 然后把这些子实例的解组合起来, 得到原问题实例的解;



6.2 分治范式

一般来说分治范式由以下的步骤组成

- **划分步骤**。在算法的这个步骤中，输入被分成 $p \geq 1$ 个部分，每个子实例的规模严格小于 n ， n 是原始实例的规模。尽管比2大的其他小的常数很普遍，但 P 的最常用的值是2。
- **治理步骤**。如果问题的规模大于某个预定的**阈值 n** ，这个步骤由执行 P 个递归调用组成，阈值是由算法的数学分析导出的
- **组合步骤**。这个步骤是组合 P 个递归调用的解来得到期望的输出值。



一般而言，分治算法有以下形式

- 如果实例 I 的规模是“小”的，则使用直接的方法求解问题并返回其答案，否则继续做下一步。
- 把实例 I 分割成 P 个大小几乎相同的子实例 $I_1, I_2, \dots, I_p$ 。
- 对每个子实例 $I_j, 1 \leq j \leq p$, 递归调用算法，并得到 P 个部分解。
- 组合 P 个部分解的结果得到原实例 I 的解，返回实例 I 的解。



寻找最大值和最小值

? 问题描述

- 在一个整数组A[1..n]中，同时寻找最大值和最小值。为了简化问题，不妨假定n是2的整数幂。
- 一种直接的算法如下

```
1.  $x \leftarrow A[1]; \quad y \leftarrow A[1]$   
2. for  $i \leftarrow 2$  to  $n$   
3.   if  $A[i] < x$  then  $x \leftarrow A[i]$   
4.   if  $A[i] > y$  then  $y \leftarrow A[i]$   
5. end for  
6. return (x,y)
```

显然，由此方法
执行的元素比较
次数是 $2n-2$

? 是否存在更有效的算法?



分治方法

- 基本思想

- 将数组分割成两半, $A[1..n/2]$ 和 $A[(n/2) + 1..n]$;
- 在每一半中找到最大值和最小值;
- 并返回这两个最小值中的最小值及这两个最大值中的最大值;



算法6.1 MINMAX

输入: n 个整数元素的数组 $A[1..n]$, n 为2的幂。

输出: (x,y) : A 中的最大元素和最小元素。

1. $\text{minmax}(1,n)$

过程 $\text{minmax}(\text{low}, \text{high})$

1. **if** $\text{high}-\text{low}=1$ **then** {数组只有2个值的情况}
2. **if** $A[\text{low}] < A[\text{high}]$ **then return** $(A[\text{low}], A[\text{high}])$
3. **else return** $(A[\text{high}], A[\text{low}])$
4. **end if**
5. **else**
6. $\text{mid} \leftarrow \lfloor (\text{low} + \text{high}) / 2 \rfloor$
7. $(x1, y1) \leftarrow \text{minmax}(\text{low}, \text{mid})$
8. $(x2, y2) \leftarrow \text{minmax}(\text{mid} + 1, \text{high})$
9. $x \leftarrow \min(x1, x2)$
10. $y \leftarrow \max(y1, y2)$
11. **return** (x, y)
12. **end if**

华中农业大学信息学院



算法分析

- 设 $C(n)$ 表示算法在由 n 个元素(n 是2的整数幂)组成的数组上执行的元素比较次数。

- 见P104
$$C(n) = \begin{cases} 1 & \text{if } n = 2 \\ 2C(n/2) + 2 & \text{if } n > 2 \end{cases}$$



定理 6.1 设数组 $A[1..n]$ 包含 n 个元素，其中 n 是2的幂，则仅用 $3n/2 - 2$ 次元素比较就能够在数组 A 中找出最大和最小元素。



6.2 二分搜索

? 问题描述

- 在数组A[1..n]中搜索已知元素x

• 基本思想

将一个给定的元素x与一个已排序数组A[low..high].的中间元素做比较

- 如果 $x < A[mid]$, 这里 $mid = \lfloor (low + high)/2 \rfloor$, 则不考虑A[mid...high], 而对A[low...mid-1]重复实施相同的方法。
- 类似地, 如果 $x > A[mid]$, 则放弃A[low...mid], 而对A[mid+1...high]重复实施相同的方法。



算法6.2 BINARYSEARCHREC

输入: 按非降序排列的 n 个元素的数组 $A[1..n]$ 和元素 x 。

输出: 如果 $x=A[j]$, $1 \leq j \leq n$, 则输出 j ; 否则输出0

1. *binarysearch*(1, n)

过程 *binarysearch*(*low*,*high*)

1. **if** *low*>*high* **then return** 0

2. **else**

3. $mid \leftarrow \lfloor (low+high)/2 \rfloor$

4. **if** $x=A[mid]$ **then return** *mid*

5. **else if** $x < A[mid]$ **then return** *binarysearch*(*low*,*mid*-1)

6. **else return** *binarysearch*(*mid*+1,*high*)

7. **end if**

华中农业大学信息学院



算法分析



假设

为了求出算法的执行时间，我们计算元素间的比较次数，因为这是一个基本运算，也就是说，算法的执行时间与完成元素间比较次数是成比例的(见1.11.2节)。

我们将假定每个三向比较当做一次比较来计算

推导见P113



定理6.2

算法BINARYSEARCHREC在 n 个元素组成的数组中搜索某个元素所执行的元素比较次数不超过 $\lfloor \log n \rfloor + 1$ 。算法BINARYSEARCHREC的时间复杂性是 $O(\log n)$ 。

算法递归深度为 $O(\log n)$ ，并且由于每个递归层次需要 $\Theta(1)$ 的空间，本算法所需的空间总量是 $\Theta(\log n)$ 。和递归算法相反，其迭代形式算法仅需要 $\Theta(1)$ 空间



6.3 合并排序

- BOTTOMUPSORT :
 - 在每一层中，我们有已排序的序列对，同时它们两两被合并而得到较大的排序序列。
 - 沿着树一层一层向上继续这个过程，直到到达根为止，这最后的序列是已经排序好的。

❓ 让我们从反向来考虑，是否能用自顶向下取代自底向上？

合并排序

基本思想

- 首先，将该数组分成两个各一半元素的数组

$A[1 \cdots 8] =$

9	4	5	2	1	7	4	6
---	---	---	---	---	---	---	---



9	4	5	2
---	---	---	---

 和

1	7	4	6
---	---	---	---

- 接着分别对这两个数组的元素排序
- 然后只是将它们合并来得到所希望的排序数组
- 至于对每一半子数组，可随意使用任何排序算法来对这两个子数组排序。
- 特别是可以使用算法SORT本身。
- 如果这样，我们实际上得出了著名的算法MERGESORT，此算法的完整描述见下面。



算法6.3 Mergesort

输入: n 个元素的数组 $A[1..n]$ 。

输出: 按非降序排列的数组 $A[1..n]$ 。

1. *mergesort*($A, 1, n$)

过程 *mergesort*($A, low, high$)

1. **if** $low < high$ **then**

2. $mid \leftarrow \lfloor (low + high) / 2 \rfloor$

3. *mergesort*(A, low, mid)

4. *mergesort*($A, mid + 1, high$)

5. *MERGE*($A, low, mid, high$)

6. **end if**



6.3.1 算法的操作过程?

- 算法MERGESORT对下列数组A排序的操作过程:
 $A=\{9,4,5,2,1,7,4,6\}$.
- 定义
 - mergesort(A, low, high) 为 MS(low, high).
 - Merge(A, low, mid, high) 为 M(low, mid, high).



MERGESORT算法过程

$MS(1,8)$	9	4	5	2	1	7	4	6
$M(1,4,8)$	1	2	4	4	5	6	7	9

$MS(1,4)$	9	4	5	2	$MS(5,8)$	1	7	4	6
$M(1,2,4)$	2	4	5	9	$M(5,6,8)$	1	4	6	7

$MS(1,2)$	9	4	$MS(3,4)$	5	2	$MS(5,6)$	1	7	4	6	$MS(7,8)$
$M(1,1,2)$	4	9	$M(3,3,4)$	2	5	$M(5,5,6)$	1	7	4	6	$M(7,7,8)$

$MS(1,1)$	9	$MS(2,2)$	4	$MS(3,3)$	5	$MS(4,4)$	2	$MS(5,5)$	1	$MS(6,6)$	7	$MS(7,7)$	4	6	$MS(8,8)$
$M(1,1,1)$	9	$M(2,2,2)$	4	$M(3,3,3)$	5	$M(4,4,4)$	2	$M(5,5,5)$	1	$M(6,6,6)$	7	$M(7,7,7)$	4	6	$M(8,8,8)$

Success!

华中农业大学



How the algorithm works?

- 这个调用链对应于树的前序遍历:访问根、左子树和右子树。
- 计算却对应于树的后序遍历:访问左子树,右子树,最后是根。
- 为了实现这个遍历,用一个栈来存储每次调用过程的局部数据。
- 在算法BOTPOMUPSORT中,合并是一层接一层进行的;而在算法MERGESORT中,合并是以后序遍历的方式进行的。两种算法的区别仅在于合并的次序,因此,当 n 是2的幂时,由算法MERGESORT执行的比较次数和算法BOTPOMUPSORT执行的次数相同。



6.3.2 合并算法分析

- 基本运算: 元素比较。
- 为了简单, 假定 n 是2的幂。
 - 如果 $n=1$, 则算法不执行任何元素的比较, 比较次数是0。
 - 如果 $n \geq 2$, 则将问题分解为(whose size is n)2个子问题(whose size are $n/2$)。
 - 合并两个子数组所需的元素比较次数在 $n/2$ 与 $n-1$ 之间。这样算法所需的**最小比较次数**:

$$c(n) = \begin{cases} 0 & \text{if } n=1 \\ 2c(n/2) + n/2 & \text{if } n \geq 2 \end{cases} \quad c(n) = \frac{n \log n}{2}$$



合并算法分析

- 最大比较次数:

$$c(n) = \begin{cases} 0 & \text{if } n=1 \\ 2c(n/2)+n-1 & \text{if } n \geq 2 \end{cases}$$

$$c(n) = 2c(n/2) + n - 1$$

$$= 2(2c(n/2^2) + n/2 - 1) + n - 1$$

$$= 2^2 c(n/2^2) + n - 2 + n - 1$$

$$= 2^2 c(n/2^2) + 2n - 2 - 1$$

.....

$$= 2^k c(n/2^k) + kn - 2^{k-1} - 2^{k-2} - \dots - 2 - 1$$

$$= 2^k c(1) + kn - \sum_{j=0}^{k-1} 2^j$$

$$= 2^k \times 0 + kn - (2^k - 1)$$

$$= kn - 2^k + 1$$

$$= n \log n - n + 1$$

华中农业大学信息学院

合并算法分析

- 如果 n 是任意的正整数(不必是2的幂), 对于由算法MERGESORT执行的元素比较次数:

$$c(n) = \begin{cases} 0 & \text{if } n = 1 \\ c(\lfloor n/2 \rfloor) + c(\lceil n/2 \rceil) + bn & \text{if } n \geq 2 \end{cases}$$
$$c(n) = \Theta(n \log n)$$



定理6.3 算法MERGESORT对一个 n 个元素的数组排序所需的时间是 $\Theta(n \log n)$, 空间是 $\Theta(n)$ 。



6.4 快速排序

- 快速排序是另外一种排序算法,该排序算法具有 $\Theta(n \log n)$ 的平均运行时间
- 这个算法优于MERGESORT算法的一点是它是在原位置上排序,即对于被排序的元素,不需要辅助的存储空间。



6.4.1 划分算法

- 基本思想

- 设 $A[\text{low}..\text{high}]$ 是一个包含 n 个数的数组，并且 $x=A[\text{low}]$
- 我们考虑重新安排数组 A 中的元素的问题，使得小于或等于 x 的元素在 x 的前面，随后 x 又在所有大于它的元素的前面
- 经过数组中元素改变排列后，对于某个 w ， $\text{low} \leq w \leq \text{high}$ ， x 在 $A[w]$ 中。
- 例如，如果 $A=\{5,3,9,2,7,1,8\}$ ；
则经过重新排列其中的元素后，得到 $A=\{1,3,2,5,7,9,8\}$
其中 $w=4$ 。



划分算法

- 这种重排列行动也称为围绕x的**拆分或划分**，x称为**主元或拆分元素**。
- 定义6.1
 - 如果元素 $A[j]$ 既不小于 $A[\text{low}..j-1]$ 中的元素，也不大于 $A[j+1..\text{high}]$ 中的元素，则称其处在一个**适当位置或正确位置**。
- 观察结论6.2
 - 用 $x(x \in A)$ 作为主元划分数组A后，x将处于一个正确位置。



算法6.5 SPLIT

输入: 数组A[low..high].

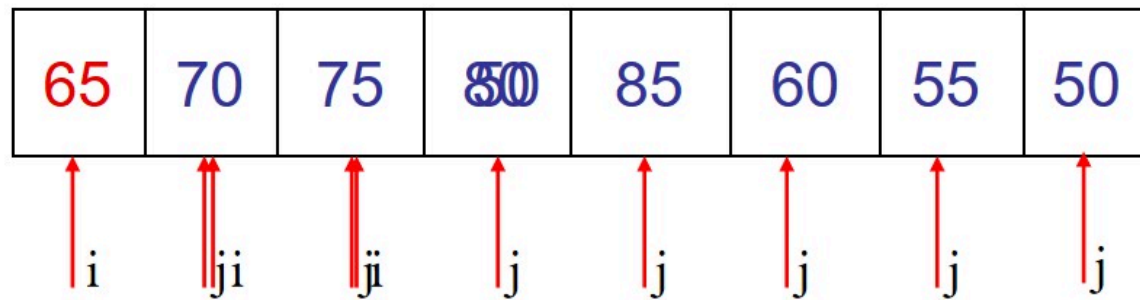
输出: (1)如有必要, 输出按上述描述的重新排列的数组A。

(2)划分元素A[low]的新位置w。

1. $i \leftarrow \text{low}$
2. $x \leftarrow A[\text{low}]$
3. **for** $j \leftarrow \text{low}+1$ **to** high
4. **if** $A[j] \leq x$ **then**
5. $i \leftarrow i+1$
6. **if** $i \neq j$ **then** 互换A[i] and A[j]
7. **end if**
8. **end for**
9. 互换A[low] and A[i]
10. $w \leftarrow i$
11. **return** A and w

华中农业大学信息学院

例子



$x =$ 65

Success!



算法分析

观察结论6.3

- 由算法SPLIT执行的元素比较次数恰好是 $n-1$ ，这样它的
时间复杂性为 $\Theta(n)$ 。

最后注意到算法仅仅用到一个额外空间用于存储它的
局部变量，因此算法的空间复杂性为 $\Theta(1)$ 。



6.4.2 快速排序算法

基本思想

- 要排序的元素 $A[\text{low}..\text{high}]$ 通过算法**SPLIT**重新排列元素, 使得原先在 $A[\text{low}]$ 中的主元占据其正确的位置 $A[w]$, 并且所有小于或等于 $A[w]$ 的元素所处的位置为 $A[\text{low}..w-1]$, 而所有大于 $A[w]$ 的元素所处的位置是 $A[w+1..\text{high}]$ 。
- 子数组 $A[\text{low}..w-1]$ 和 $A[w+1..\text{high}]$ **递归地**排序, 自动产生整个排序数组



算法6.6 QIUCKSORT

输入: n 个元素的数组 $A[1..n]$

输出: 按非降序排列的数组 A 中的元素

1. *quicksort*($A, 1, n$)

过程 *quicksort*($A, low, high$)

1. **if** $low < high$ **then**

2. $w = \text{SPLIT}(A[low..high])$ { w 为 $A[low]$ 的
新位置}

3. *quicksort*($A, low, w-1$)

4. *quicksort*($A, w+1, high$)

5. **end if**

华中农业大学信息学院

例子

假设数组 $A = \{4, 6, 3, 1, 8, 7, 2, 5\}$.

4	6	3	1	8	7	2	5
2	3	1	4	8	7	6	5
2	3	1		8	7	6	5
1	2	3		5	7	6	8
1		3		5	7	6	
1		3		5	7	6	
					7	6	
					6	7	
					6		
					6		

Success!

华中农业大学信息学院



MERGESORT 和 QUICKSORT

- 算法SPLIT和算法QUICKSORT的关系类似于算法MERGE和算法MERGESORT的关系。
- 两个排序算法都由名为SPLIT和MERGE的算法之一的一系列调用组成。
- 在算法MERGESORT中，合并排序序列属于组合步骤，而在算法QUICKSORT中的划分过程则属于划分步骤。事实上，在算法QUICKSORT中组合步骤是不存在的。



算法6.6 QUICKSORT

输入: n 个元素的数组 $A[1..n]$

输出: 按非降序排列的数组 A 中的元素

1. *quicksort*($A, 1, n$)

过程 *quicksort*($A, low, high$)

1. **if** $low < high$ **then**

2. $w = SPLIT(A[low..high])$ { w 为 $A[low]$ 的
新位置}

3. *quicksort*($A, low, w-1$)

4. *quicksort*($A, w+1, high$)

5. **end if**

华中农业大学信息学院



6.4.3 快速排序算法分析

- 空间复杂度:
 - 虽然算法不需要辅助空间存储数组元素，但算法还是至少需要 $\Theta(n)$ 的空间，**工作空间在 $\Theta(\log n)$ 和 $\Theta(n)$ 之间变化**。这是因为在每次递归调用中，表示下一次要排序的数组右边部分的左右界 $w+1$ 和 $high$ 必须保存。



6.4.3.1 最坏情况的行为



定理6.5

- 在最坏的情况下，算法QUICKSORT的运行时间是 $\Theta(n^2)$ ，然而如果总是选择中项作为主元，它的时间复杂性是 $\Theta(n \log n)$ 。
- 推导见P116



6.4.3.2 平均情况的行为



假设

- 假定对于数1, 2, ..., n的i个排列中的每一个排列的出现是等可能性的, 这一点确保了在数组中的每个数以同样可能作为第一个元素, 并被选为主元, 也就是说, 数组A中的任意元素被选为主元的概率是1/n.
- 设C(n)表示对大小为n的输入数据算法所做的平均比较次数, 根据观察结论6.3, 得到:
- 步骤2恰好耗费n-1次比较;
- 步骤3和步骤4分别耗费了C(w-1)和C(n-w)次比较;
- 因此总的比较次数为:

$$C(n) = (n-1) + \frac{1}{n} \sum_{w=1}^n (C(w-1) + C(n-w))$$

华中农业大学信息学院



6.4.3.2 平均情况的行为

- 因为
$$\sum_{w=1}^n C(n-w) = C(n-1) + C(n-2) + \dots + C(0) = \sum_{w=1}^n C(w-1)$$

- 因此
$$C(n) = (n-1) + \frac{2}{n} \sum_{w=1}^n C(w-1)$$

$$nC(n) = n(n-1) + 2 \sum_{w=1}^n C(w-1)$$

- 如果用n-1代替n，得到：

$$(n-1)C(n-1) = (n-1)(n-2) + 2 \sum_{w=1}^{n-1} C(w-1)$$

- 推导见P117



6.4.3.2 平均情况的行为

- 等式6.3减去等式6.4，再重新排列各项得到

$$\frac{C(n)}{n+1} = \frac{C(n-1)}{n} + \frac{2(n-1)}{n(n+1)}$$

- 现在把变量更换为一个新的变量D， $D(n) = \frac{C(n)}{n+1}$

- 那么， $D(n) = D(n-1) + \frac{2(n-1)}{n(n+1)}$ ， $D(1) = 0$.

- 其解为
$$D(n) = 2 \sum_{j=1}^n \frac{j-1}{j(j+1)}$$

6.4.3.2 平均情况的行为

- 因为
$$\begin{aligned} 2 \sum_{j=1}^n \frac{j-1}{j(j+1)} &= 2 \sum_{j=1}^n \frac{2}{(j+1)} - 2 \sum_{j=1}^n \frac{1}{j} \\ &= 4 \sum_{j=2}^{n+1} \frac{1}{j} - 2 \sum_{j=1}^n \frac{1}{j} \\ &= 2 \sum_{j=1}^n \frac{1}{j} - \frac{4n}{n+1} \\ &= 2 \ln n - \Theta(1) \quad (\text{Eq. 2.16}) \\ &= \frac{2}{\log e} \log n - \Theta(1) \approx 1.44 \log n \end{aligned}$$

- 结果,
$$C(n) = (n+1)D(n) \approx 1.44n \log n$$



定理6.6: 算法QUICKSORT对n个元素的数组进行排序执行的平均比较次数是 $\Theta(n \log n)$ 。



课堂练习

给定 n 个不等的整数构成的集合 L 和整数 s ，设计一个算法判断在 L 中是否存在两个整数 x 和 y ($x < y$)，满足 $x + y = s$ 。
以加法运算作为基本运算分析你的算法在最坏下的时间复杂度



6.5 寻找中项和第k小元素

? 选择问题

- 在一个包含n个元素的集合中寻找第k个最小元素。(中值是第 $\lceil n/2 \rceil$ 个最小元素)

- 直接方法:

- 对所有的元素排序并取出中间一个元素, 这个方法需要 $\Omega(n \log n)$ 时间, 因为任何基于比较的排序过程在最坏的情况下必须至少要花费这么多时间

? Can we find an efficient method in optimal linear time?

能否在最优线性时间内找到更快的算法?



SELECT 算法

- 假设递归算法中，在每个递归调用的划分步骤后，我们丢弃元素的一个固定部分并且对剩余的元素递归。则问题的规模以几何级数递减，也就是在每个调用过程中，问题的规模以一个常因子被减小。
- 假设某个算法丢弃1/3并对剩余的2/3部分递归，那么在第二次调用中，元素的数目变为 $2n/3$ 个，第三次调用中为 $4n/9$ ，第四次调用中变为 $8n/27$ ，等等。
- 现在假定在每次调用中，算法对每个元素耗费的时间不超过一个常数，则耗费在处理所有元素上的全部时间产生一个几何级数：

$$cn + (2/3)cn + (2/3)^2 cn + \dots + (2/3)^j cn + \dots$$

这里。 c 是某个被适当选择的常数，由等式2.12，这个总数小于

$$\sum_{j=0}^{\infty} cn(2/3)^j = 3cn = \Theta(n)$$

SELECT 算法



基本思想

- 首先，如果元素个数小于预定义的**阈值**44，则算法使用直接的方法计算第k小的元素，至于如何选择阈值在以后的算法分析中将会清楚。
- 下一步把n个元素**划分**成 $\lfloor n/5 \rfloor$ 组，每组由5个元素组成，如果n不是5的倍数，则排除剩余的元素，这应当不影响算法的执行。每组进行**排序**并取出它的中项即第三个元素
- 接着将这些中项序列中的中项元素记为**mm**，它是通过递归计算得到的。将数组A中的元素划分成三个数组: A1, A2, A3，其中分别包含小于、等于和大于mm的元素。
- 最后，求出第k小的元素出现在三个数组中的哪一个，并根据测试结果，算法或者返回第k小的元素，或者在A1 or A3上递归。



算法6.4 SELECT

输入: n 个元素的数组 $A[1..n]$ 和整数 $k, 1 \leq k \leq n$ 。

输出: A 中的第 k 小元素。

1. $\text{select}(A, 1, n, k)$

过程 $\text{select}(A, \text{low}, \text{high}, k)$

1. $p \leftarrow \text{high} - \text{low} + 1$

2. **if** $p < 44$ **then** 将 A 排序 **return** $(A[k])$

3. Let $q = \lfloor p/5 \rfloor$. 将 A 分成9组, 每组5个元素。如果5不整除 p , 则排除剩余的元素

4. 将 q 组中的每一组单独排序, 找出中项。所有中项的集合为 M

5. $mm \leftarrow \text{select}(M, 1, q, \lceil q/2 \rceil)$ { mm 为中项集合的中项}

6. 将 $A[\text{low}..\text{high}]$ 分成三组:

$A1 = \{a | a < mm\}$ $A2 = \{a | a = mm\}$ $A3 = \{a | a > mm\}$

7. **case**

$|A1| \geq k$: **return** $\text{select}(A1, 1, |A1|, k)$

$|A1| + |A2| \geq k$: **return** mm

$|A1| + |A2| < k$: **return** $\text{select}(A3, 1, |A3|, k - |A1| - |A2|)$

8. **end case**

华中农业大学信息学院



例子

- 为方便起见，让我们暂时将算法中的阈值44改为一个较小的数6

 假设存在数组A[1..25]:

- 8, 33, 17, 51, 57, 49, 35, 11, 25, 37, 14, 3, 2, 13, 32, 12, 6, 29, 32, 54, 5, 16, 22, 23, 7

我们要在数组A中找到第13个最小的元素



例子

8	33	17	51	57	49	35	11	25	37	14	3	2	13	52	12	6	29	32	54	5	16	22	23	7
---	----	----	----	----	----	----	----	----	----	----	---	---	----	----	----	---	----	----	----	---	----	----	----	---

partition

8	33	17	51	57	49	35	11	25	37	14	3	2	13	52	12	6	29	32	54	5	16	22	23	7
---	----	----	----	----	----	----	----	----	----	----	---	---	----	----	----	---	----	----	----	---	----	----	----	---

sort

8	17	33	51	57	11	25	35	37	49	2	3	13	14	52	6	12	29	32	54	5	7	16	22	23
---	----	----	----	----	----	----	----	----	----	---	---	----	----	----	---	----	----	----	----	---	---	----	----	----

median

13	16	29	33	35
----	----	----	----	----

partition

8	17	11	25	14	3	2	13	12	6	5	16	22	23	7	29	33	51	57	49	35	37	52	32	54
---	----	----	----	----	---	---	----	----	---	---	----	----	----	---	----	----	----	----	----	----	----	----	----	----

discard

华中农业大学信息学院



例子

$A[1...15]$

8	17	11	25	14	3	2	13	12	6	5	16	22	23	7
---	----	----	----	----	---	---	----	----	---	---	----	----	----	---

partition

8	17	11	25	14
---	----	----	----	----

3	2	13	12	6
---	---	----	----	---

5	16	22	23	7
---	----	----	----	---

sort

8	11	14	17	25
---	----	----	----	----

2	3	6	12	13
---	---	---	----	----

5	7	16	22	23
---	---	----	----	----

median

6	14	16
---	----	----

partition

8	11	3	2	13	12	6	5	7
---	----	---	---	----	----	---	---	---

14

17	25	16	22	23
----	----	----	----	----

discard



例子

$A[11...15]$

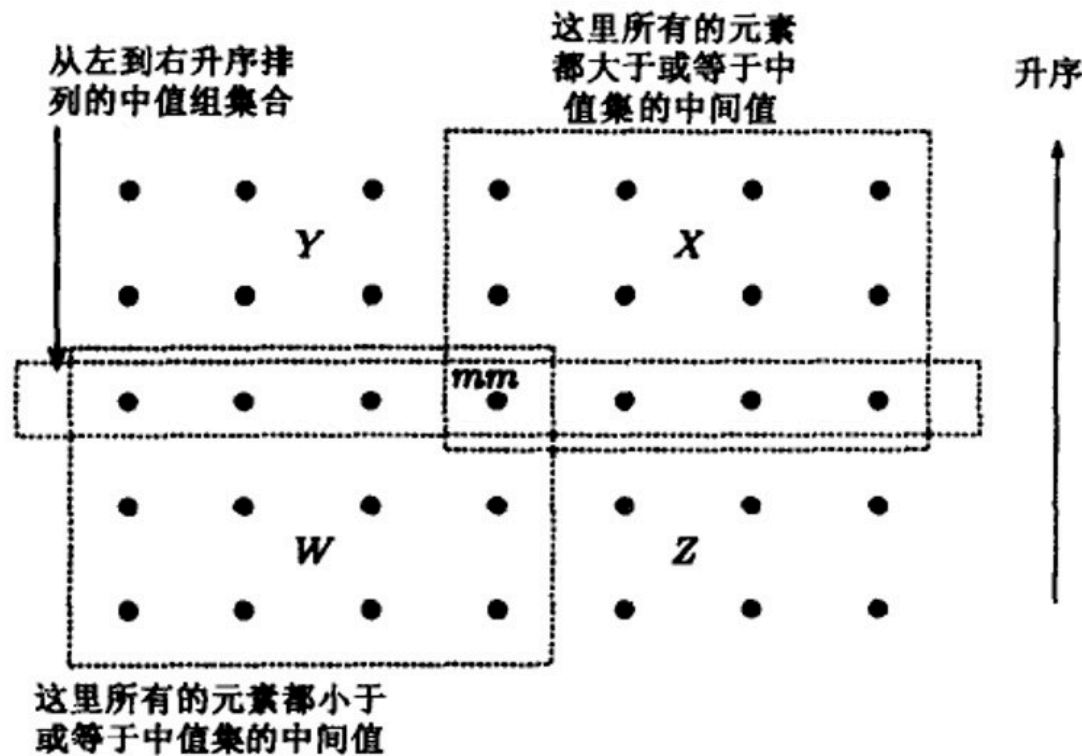
17	25	16	22	23
----	----	----	----	----

sort

16	17	22	23	25
----	----	----	----	----

Succeed! The result is $A[13]=22$

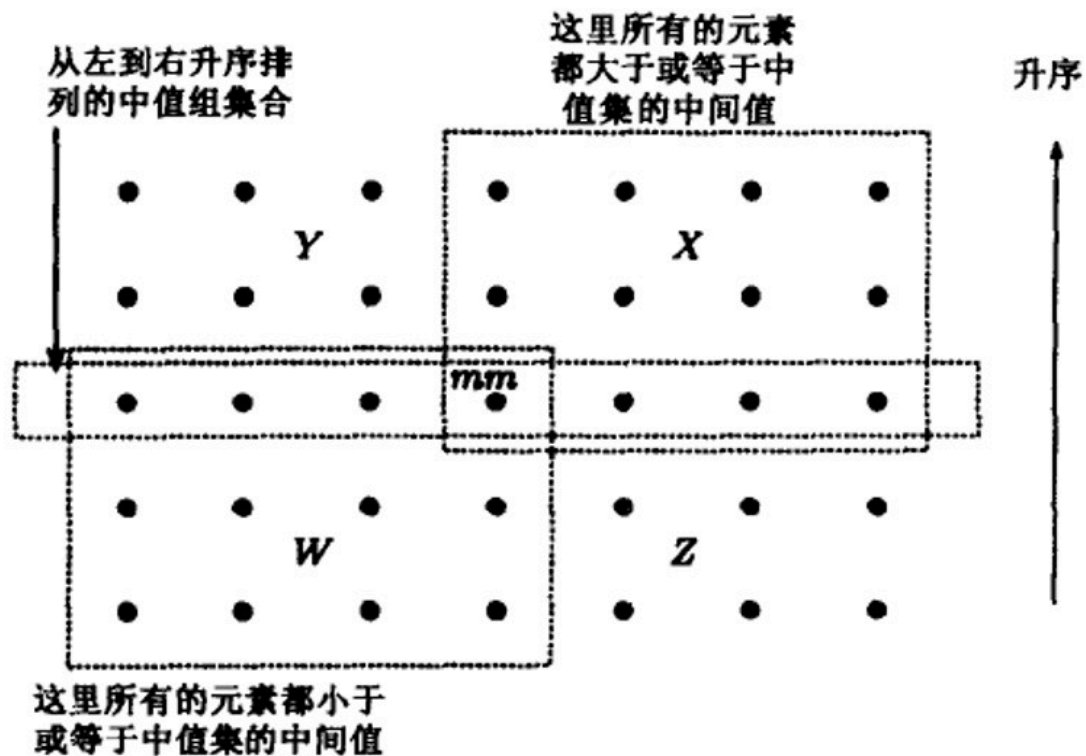
selection 算法分析



其中的元素已被划分成具有5元素的各个组，每组元素从底到顶按升序排列。

而且这些组以这样的方式整齐排列：它们的中项是从左到右以升序排列的。

selection算法分析



所有围在矩形W中的元素是小于或等于mm的，所有围在矩形X中的元素是大于或等于mm的。

设 A'_1 表示小于或等于mm的元素集， A'_3 是大于或等于mm的元素集
这个算法中， A_1 是严格小于mm的元素集， A_3 是严格大于mm的元素集



- 由于 A'_1 至少与 w 同样大, 我们有

$$|A'_1| \geq 3\lceil [n/5]/2 \rceil \geq \frac{3}{2}\lfloor n/5 \rfloor$$

- 因此 $|A_3| \leq n - \frac{3}{2}\lfloor n/5 \rfloor \leq n - \frac{3}{2}\left(\frac{n-4}{5}\right) = n - 0.3n + 1.2 = 0.7n + 1.2.$

- 由参数的对称性得到

$$|A'_3| \geq \frac{3}{2}\lfloor n/5 \rfloor \text{ and } |A_1| \leq 0.7n + 1.2$$

- 这样就为在 A_1 和 A_3 的元素个数建立了上界:
 - 即小于 mm 的元素的个数和大于 mm 的元素个数不能够超过大约 $0.7n$ (n 的常分数倍)。



时间复杂度

算法的运行时间 $T(n)$ P112

- 在算法中select过程的步骤1和步骤2耗费的时间都是 $\Theta(1)$
- 步骤3为 $\Theta(n)$ 时间。由于对每组排序需要时间为常量，步骤4耗费 $\Theta(n)$ 时间，
- 步骤5所需的时间为 $T(\lfloor n/5 \rfloor)$ ，步骤6需要 $\Theta(n)$ 时间。由以上的分析，步骤7耗费的时间至多是 $T(0.7n+1.2)$ 。
- 当 $n \geq 44$ 时，如果 $0.7n + 1.2 \leq 0.75n - 1$ 这个不等式将成立。得出的结论为，对于 $n \geq 44$ ，步骤7所耗费的时间至多是 $T(\lfloor 0.75n \rfloor)$ 。



时间复杂度

- 这个分析蕴含着算法SELECT的运行时间有以下的递推式

$$T(n) \leq \begin{cases} c & \text{if } n < 44 \\ T(\lfloor n/5 \rfloor) + T(\lfloor 3n/4 \rfloor) + cn & \text{if } n \geq 44 \end{cases}$$

$$T(n) \leq \frac{cn}{1-1/5-3/4} = 20cn$$



定理6.4

- 在一个由n个元素组成的线序集合中提取第k小元素，所需的时间是 $\Theta(n)$ ，特别地，n个元素的中值可以在 $\Theta(n)$ 时间找出。



6.7 大整数乘法

- 我们假定大小一定的整数相乘需要耗费定量的时间，而当两个任意长的整数相乘时，这个假定就不再有效了
- 对于处理不定长数的算法的数据输入，通常是按二进制的位数来计算的(或按二进制数字来计算)
- 设 u 和 v 是两个 n 位的整数，传统的乘法算法需要 $\Theta(n^2)$ 数字相乘来计算 u 和 v 的乘积。

? Is there much more efficient method?



6.7 大整数乘法

- 设 u 和 v 都是 n 位的二进制整数，现在要计算它们的乘积 $u*v$ 。
- 我们可以用小学所学的方法来设计一个计算乘积 $u*v$ 的算法，但是这样做计算步骤太多，显得效率较低。如果将每2个1位数的乘法或加法看作一步运算，那么这种方法要作 $O(n^2)$ 步运算才能求出乘积 $u*v$ 。
- 下面我们用分治法来设计一个更有效的大整数乘积算法。
- 我们将 n 位的二进制整数 u 和 v 各分为2段，每段的长为 $n/2$ 位(为简单起见，假设 n 是2的幂)，



分治技术

假定 n 是2的幂

$$\begin{array}{l} u = w2^{n/2} + x : \\ v = y2^{n/2} + z : \end{array} \begin{array}{|c|c|} \hline \overbrace{}^{n/2} & \overbrace{}^{n/2} \\ \hline w & x \\ \hline y & z \\ \hline \end{array}$$

u 和 v 的乘积可以计算为:

$$uv = (w2^{n/2} + x)(y2^{n/2} + z) = wy2^n + (xy + wz)2^{n/2} + xz$$

时间复杂度函数为:

$$T(n) = \begin{cases} d & \text{if } n = 1 \\ 4T(n/2) + bn & \text{if } n > 1 \end{cases}$$

递推式的解:

$$T(n) = \Theta(n^2)$$



分治技术

现在考虑:

$$wz + xy = (w + x)(y + z) - wy - xz$$

我们得到:

$$uv = wy2^n + ((w + x)(y + z) - wy - xz)2^{n/2} + xz$$

时间复杂度是:

$$T(n) = \begin{cases} d & \text{if } n = 1 \\ 3T(n/2) + bn & \text{if } n > 1 \end{cases}$$

wy,xz不用再做二次计算

递推式的解是:

$$T(n) = \Theta(n^{\log 3}) = O(n^{1.59})$$

华中农业大学信息学院

课堂练习

- 给出在一个整数数组 $A[1, \dots, n]$ 中求出所有元素和的分治算法。并求算法所需的工作空间？



6.8 矩阵乘法

两个矩阵 A, B: A 的列数等于 B 的行数, 则 A、B 可以相乘。即, 如果 $A = (a_{ij})$ 是一个 $m * n$ 的矩阵, $B = (b_{jk})$ 是一个 $n * p$ 的矩阵, 则它们的乘积 $C = AB$ 是 $m * p$ 矩阵 $C = (c_{ik})$ 。

$$\begin{matrix} \begin{bmatrix} 1 & 0 & 2 \\ -1 & 3 & 1 \end{bmatrix} & \times & \begin{bmatrix} 3 & 1 \\ 2 & 1 \\ 1 & 0 \end{bmatrix} & = & \begin{bmatrix} (1 \times 3 + 0 \times 2 + 2 \times 1) & (1 \times 1 + 0 \times 1 + 2 \times 0) \\ (-1 \times 3 + 3 \times 2 + 1 \times 1) & (-1 \times 1 + 3 \times 1 + 1 \times 0) \end{bmatrix} & = & \begin{bmatrix} 5 & 1 \\ 4 & 2 \end{bmatrix} \\ A_{23} & & B_{32} & & & & C_{22} \end{matrix}$$



6.8 矩阵乘法

? 问题描述

- 令A和B是两个 $n \times n$ 的矩阵，我们希望计算它们的乘积 $C=AB$ 。



6.8.1 矩阵乘法

- 在传统方法中， $C=A \times B$ 是由以下公式计算的

$$C(i, j) = \sum_{k=1}^n A(i, k)B(k, j)$$

- 因为每计算一个元素 $C[i, j]$ ，需要做 n 个乘法和 $n-1$ 次加法
- 很容易看出算法需要 n^3 次乘法运算和 n^3-n^2 次加法运算。这导致其时间复杂度为 $\Theta(n^3)$ 。

? Is there much more efficient method?



6.8.2 递归方法

假定 $n=2^k$, $k \geq 0$, 如果 $n \geq 2$, 则A, B和C可分别分成4个大小为 $n/2 \times n/2$ 的子矩阵

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

用分治方法来计算C的组成:

$$C = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix}$$



递归方法

计算量总耗费是：

$$T(n) = \begin{cases} m & \text{if } n = 1 \\ 8T(n/2) + 4(n/2)^2 a & \text{if } n \geq 2 \end{cases}$$

最终结果：

$$T(n) = (m + \frac{a2^2}{8-2^2})n^3 - (\frac{a2^2}{8-2^2})n^2 = mn^3 + an^3 - an^2$$

观察结论 6.4

- 递归方法需要 n^3 次乘法运算和 $n^3 - n^2$ 次加法运算，这些刚好和前面传统方法所讲述的一样。

两种算法的区别仅仅在于矩阵元素相乘的次序上，其他方面是一样的

华中农业大学信息学院



6.8.3 STRASSEN算法



基本思想

- 增加加减法的次数来减少乘法次数

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \quad \text{和} \quad B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

$$C = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$



STRASSEN算法

基本思想

我们首先计算以下的乘积

$$d_1 = (a_{11} + a_{22})(b_{11} + b_{22})$$

$$d_2 = (a_{21} + a_{22})b_{11}$$

$$d_3 = a_{11}(b_{12} - b_{22})$$

$$d_4 = a_{22}(b_{21} - b_{11})$$

$$d_5 = (a_{11} + a_{12})b_{22}$$

$$d_6 = (a_{21} - a_{11})(b_{11} + b_{12})$$

$$d_7 = (a_{12} - a_{22})(b_{21} + b_{22})$$

接着从一下面式子计算出C

$$C = \begin{pmatrix} d_1 + d_4 - d_5 + d_7 & d_3 + d_5 \\ d_2 + d_4 & d_1 + d_3 - d_2 + d_6 \end{pmatrix}$$



时间复杂度

算法用到的加法是18次，乘法是7次，对于其运行时间产生以下递推式 (P121)

$$T(n) = \begin{cases} m & \text{if } n=1 \\ 7T(n/2) + 18(n/2)^2 a & \text{if } n \geq 2 \end{cases}$$

或

$$T(n) = \begin{cases} m & \text{if } n=1 \\ 7T(n/2) + (9a/2)n^2 & \text{if } n \geq 2 \end{cases}$$

$$T(n) = (m + \frac{(9a/2)2^2}{7-2^2})n^{\log 7} - (\frac{(9a/2)2^2}{7-2^2})n^2 = mn^{\log 7} + 6an^{\log 7} - 6an^2$$

即运行时间为：

$$\Theta(n^{\log 7}) = O(n^{2.81})$$

华中农业大学信息学院



时间复杂度

- Strassen算法的高效之处，就在于，它成功的减少了乘法次数，将8次乘法，减少到7次。
- 不要小看这减少的一次，和传统的递归算法比较起来，每递归计算一次，效率就可以提高 $1/8$ ，比如一个大的矩阵递归5次后， $(7/8)^5 = 0.5129$ ，效率提升一倍。不过，这只是理论值，实际情况中，有隐含开销，并不是最常用算法。
- 矩阵是稀疏矩阵时，为稀疏矩阵设计的方法更快。所以，稠密矩阵上的快速矩阵乘法实现一般采用Strassen算法。



6.8.4 三个算法的比较

三种算法所做的算术运算的次数

	乘法	加法	复杂度
传统算法.	n^3	$n^3 - n^2$	$\Theta(n^3)$
递归方法.	n^3	$n^3 - n^2$	$\Theta(n^3)$
STRASSEN 算法.	$n^{\log 7}$	$6 n^{\log 7} - 6 n^2$	$\Theta(n^{\log 7})$

华中农业大学信息学院

课堂练习

- 设 M 是一个 n 行 n 列的0-1矩阵，每行的1都排在0的前面
 - (1) 设计一个最坏情况下 $O(n \log n)$ 时间的算法找到 M 中含有1最多的行，说明算法的设计思想，估计最坏情况下的时间复杂度
 - (2) 对上述问题，能否找到一个最坏情况下 $O(n)$ 时间的算法



6.9 最近点对问题

? 问题:

- 设S是平面上n个点的集合，在这一节中，我们考虑在S中找到一个点对p和q的问题，使其相互距离最短。
- 换句话说，希望在S中找到具有这样性质的两点 $p_1 = (x_1, y_1)$ 和 $p_2 = (x_2, y_2)$ ，使它们间的距离在所有S中点对间为最小。

$$d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$



最近点对问题

- **蛮力算法**就是简单地检验所有可能的 $n(n-1)/2$ 个距离并返回最短间距的点对。耗费 $\Theta(n^2)$ 的时间
- 我们将运用分治设计技术描述一个时间复杂性为 $\Theta(n \log n)$ 的算法来解决最近点对问题。

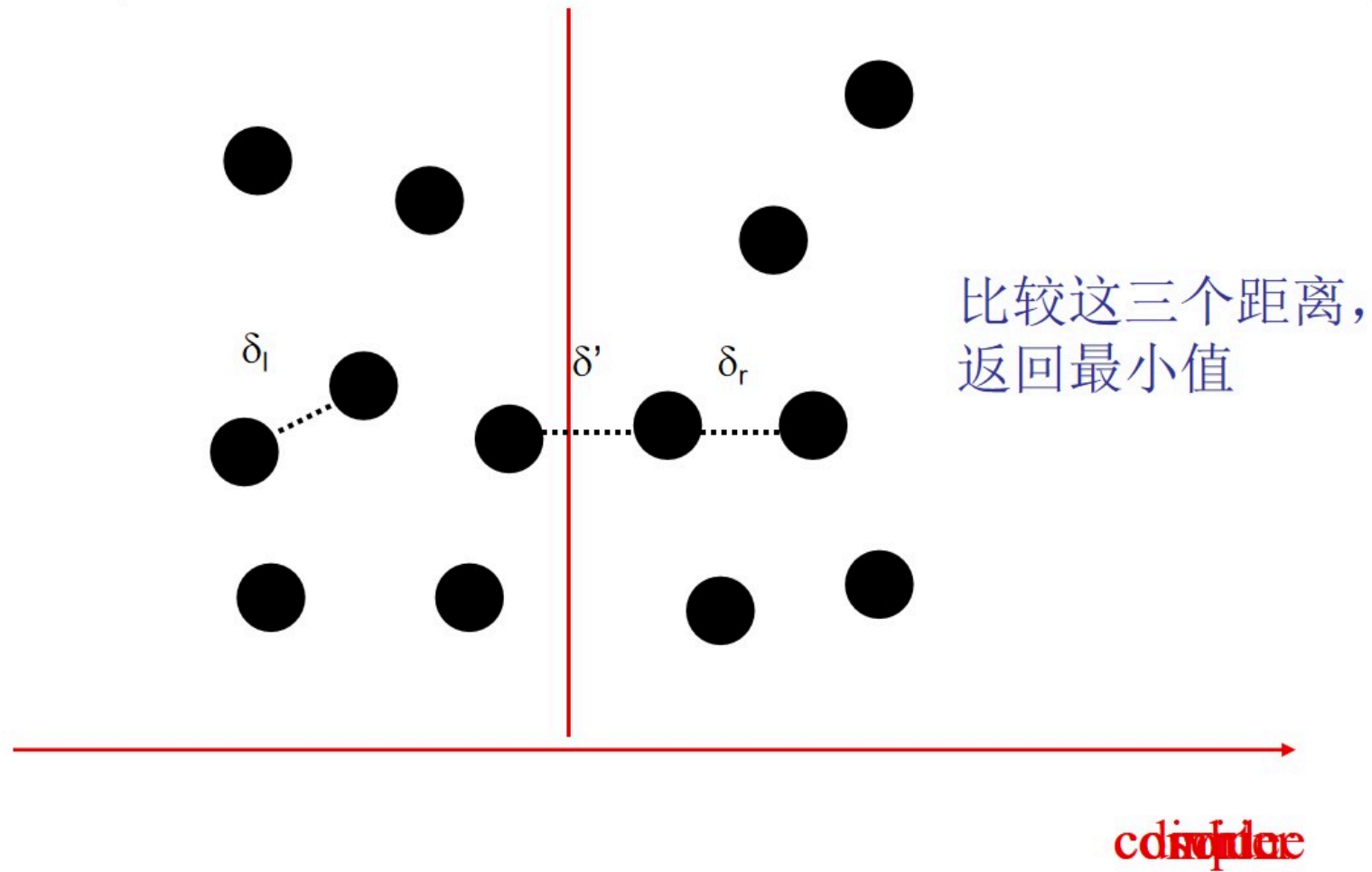


分治方法

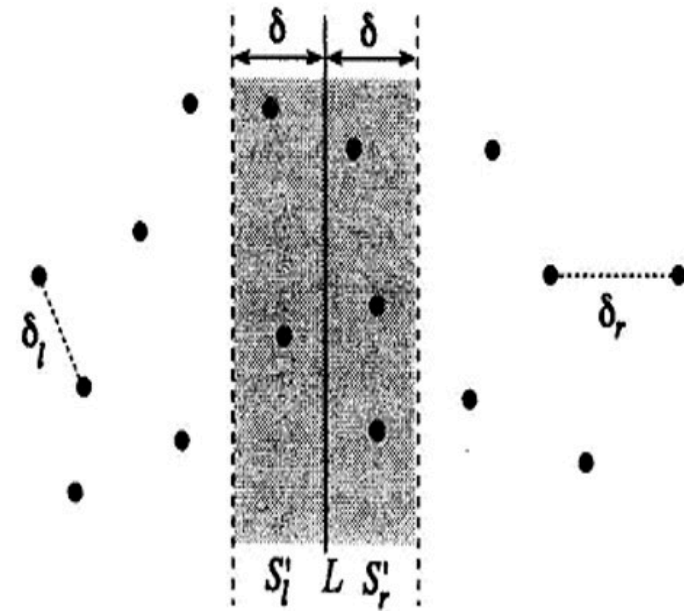
- Sort:
 - 第一步是以x坐标增序对S中的点排序
- Divide:
 - S点集被垂直线L大约划分成两个子集 S_l 和 S_r ，使 $|S_l| = \lfloor |S|/2 \rfloor$ ， $|S_r| = \lceil |S|/2 \rceil$ ，设L是经过x坐标 $S[\lfloor n/2 \rfloor]$ 的垂直线，这样在 S_l 中的所有点都落在或靠近L的左边，而所有 S_r 中的点落在或靠近L的右边。
- Conquer:
 - 现在按上述递归地进行，两个子集 S_l 和 S_r 的最小间距 δ_l 和 δ_r 可分别计算出来。
- Combine:
 - 组合步骤是把在 S_l 中的点与 S_r 中的点之间的最小间距 δ' 计算出来。最后所要求的解是 δ_l ， δ_r 和 δ' 中的最小值。

华中农业大学信息学院

基本思想

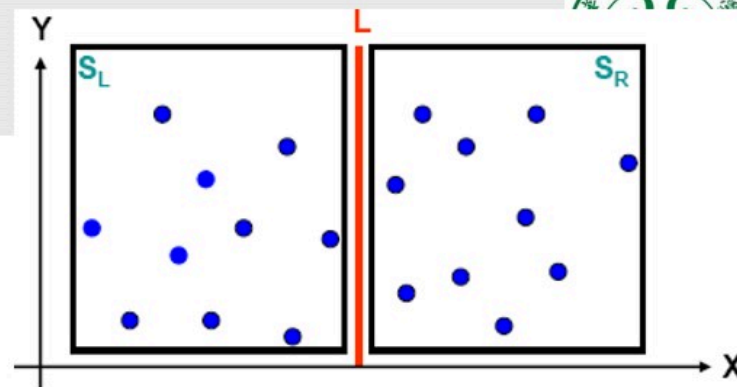


- 怎样合并结果，这一步的关键在于计算 δ' ，计算 S_l 中的每个点与 S_r 中每个点之间的距离的朴素方法需要 $\Omega(n^2)$ 。

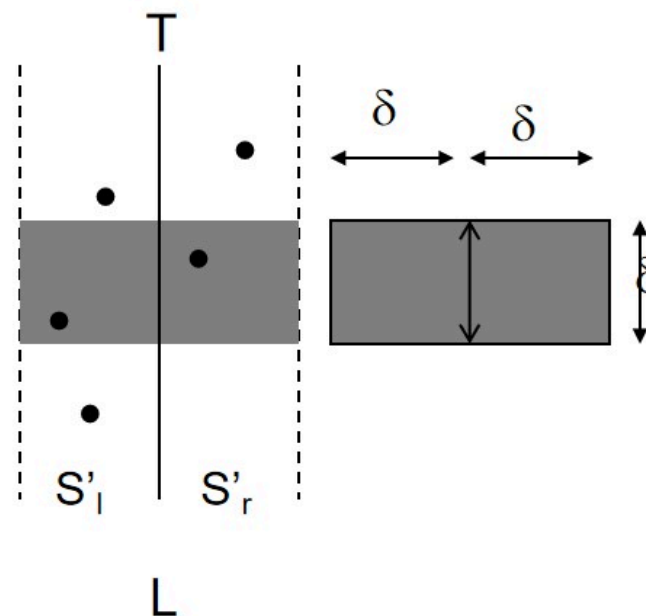


■ 设 $\delta = \min\{\delta_l, \delta_r\}$ ，如果最近点对由 S_l 中的某个点 p_l 与 S_r 中的某个点 p_r 组成，则 p_l 和 p_r 一定在划分线 L 的距离 δ 内。这样，如果令 S_l' 和 S_r' 分别表示为在线 L 距离 δ 内的 S_l 和 S_r 中的点，则 p_l 一定在 S_l' 中， p_r 一定在 S_r' 中。

- 假设 $\delta' \leq \delta$ ，则存在两点 $p_l \in S'_l$ 和 $p_r \in S'_r$ ，有 $d(p_l, p_r) = \delta'$ ，从而 p_l 和 p_r 之间的垂直距离不超过 δ



- 因为 p_l, p_r 这两点都在以垂直线 L 为中心的 $\delta \times 2\delta$ 矩形区内或其边界上
- 设 T 是两个垂直带内的点的集合
- 在 $\delta \times 2\delta$ 矩形区内，如果任意两点间的距离一定不超过 δ (因为只有小于 δ 的两点才是我们关注的)，则这个矩形最多能容纳 8 个点，其中至多 4 个点属于 S_l ，4 个点属于 S_r 。





分治方法

观察结论6.5

- T中的每个点最多需要和T中的7个点进行比较。
- 上述的观察结论仅给出与T中每个点p的比较点数的上界，而没有给出哪些点与P比较的任何信息。
- 稍想一下，即可知P一定是与T中临近的点比较。为了找到这样的相邻点，我们借助于以x坐标的递增序对T中的点重新排序，然后，不难看出只需对T中每个点P，与它们的Y坐标递增序下的7个点比较。



时间复杂度

- 排序S中的点需要 $\Theta(n \log n)$ 时间。
- 把点划分成 S_l 和 S_r 子集需要 $D(1)$ 时间，因为此时点已排序了。
- 至于组合步骤，我们可以看到它包含对T中点的排序和每点与其他至多7个点做比较，排序耗费 $O(n \log n)$ 时间，并存在至多 $7n$ 次比较，这样组合步骤最坏情况下需要 $\Theta(n \log n)$ 时间。
- 注意如果点数为3，则计算它们间最小间距只需要做三次比较，算法执行的递推关系变为：

$$T(n) = \begin{cases} 1 & \text{if } n = 2 \\ 3 & \text{if } n = 3 \\ 2T(n/2) + O(n \log n) & \text{if } n > 3 \end{cases} \rightarrow T(n) = O(n \log^2 n)$$



时间复杂度

在组合步骤中，每次需要对T排序，现在只需要在 $\Theta(n)$ 时间里从数组Y中提取它的元素，这样递推式变为：

$$T(n) = \begin{cases} 1 & \text{if } n = 2 \\ 3 & \text{if } n = 3 \\ 2T(n/2) + \Theta(n) & \text{if } n > 3 \end{cases} \quad \rightarrow T(n) = O(n \log n)$$



算法6.7 CLOSESTPAIR

输入: 平面上 n 个点的集合 S 。

输出: S 中两点的最小距离。

1. 以 x 坐标增序对 S 中的点排序。
2. 序 $Y \leftarrow$ 以 y 坐标增序对 S 中的点排序。
3. $\delta \leftarrow cp(1, n)$.

过程 $cp(low, high)$

1. **if** $high - low + 1 \leq 3$ **then** 用直接方法计算 δ
2. **else**
3. $mid \leftarrow \lfloor (low + high) / 2 \rfloor$
4. $x_0 \leftarrow x(S[mid])$
5. $\delta_l \leftarrow cp(low, mid)$
6. $\delta_r \leftarrow cp(mid + 1, high)$
7. $\delta \leftarrow \min\{\delta_l, \delta_r\}$
8. $k \leftarrow 0$

To be continued...

华中农业大学信息学院



过程 $cp(low, high)$

```
9.      for  $i \leftarrow 1$  to  $n$            {从Y中抽取T}
10.         if  $|x(Y[i]) - x_0| \leq \delta$  then
11.              $k \leftarrow k + 1$ 
12.              $T[k] \leftarrow Y[i]$ 
13.         end if
14.     end for
15.      $\delta' \leftarrow 2\delta$ 
16.     for  $i \leftarrow 1$  to  $k - 1$ 
17.         for  $j \leftarrow i + 1$  to  $\min\{i + 7, k\}$ 
18.             if  $d(T[i], T[j]) < \delta'$  then  $\delta' \leftarrow d(T[i], T[j])$ 
19.         end for
20.     end for
21.      $\delta \leftarrow \min\{\delta, \delta'\}$ 
22. end if
23. return  $\delta$ 
```

华中农业大学信息学院



CLOSESTPAIR的时间复杂度



定理6.7

- 假定有平面上 n 个点的集合 S ，算法CLOSESTPAIR在 $\Theta(n \log n)$ 时间内找到 S 中具有最小间距的一个点对。



课堂练习

- 设 $A = \{a_1, a_2, \dots, a_n\}$, $B = \{b_1, b_2, \dots, b_m\}$ 是整数集合，其中 $m = O(\log n)$. 设计一个算法找出集合 $C = A \cap B$. 给出伪码描述